

CẤU TRÚC DỮ LIỆU & GIẢI THUẬT

CÂY NHỊ PHÂN TÌM KIẾM



TS. TRẦN NGỌC VIỆT
TH.S LÊ CÔNG HIẾU



HỌC KỲ 3 – NĂM HỌC 2022-2023



KHÓA K28 & K27



NỘI DUNG

01. TỔNG QUAN

02. THAO TÁC TRÊN CT CÂY

03. CÁC LOẠI CÂY

04. CÂY NHỊ PHÂN

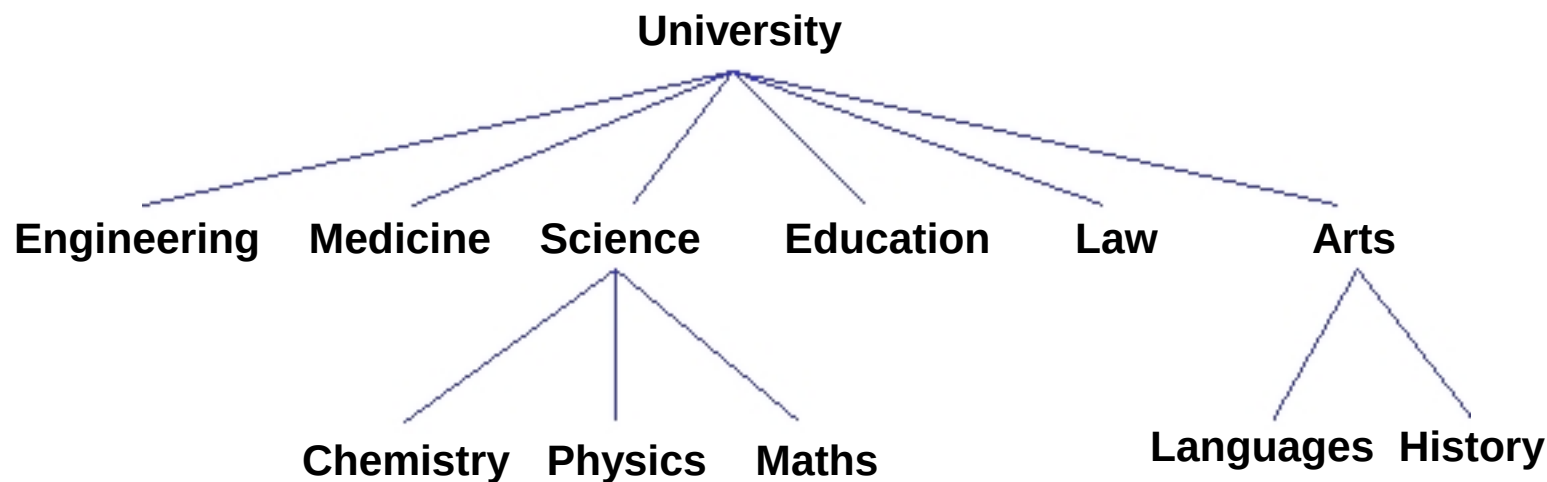
05. DUYỆT CÂY

KHÁI NIỆM CƠ BẢN

- Cây là một cấu trúc phân tầng của các phần tử gọi là nút (*node*)
 - Mỗi node chứa một phần tử đơn.
 - Mỗi phần tử có thể có 1 hoặc nhiều nhánh kết nối với các nút khác, được gọi là nút con.
- Mọi cây có 1 nút gốc – root duy nhất.
 - Mọi nút trừ nút gốc đều là con của một nút khác – nút cha.

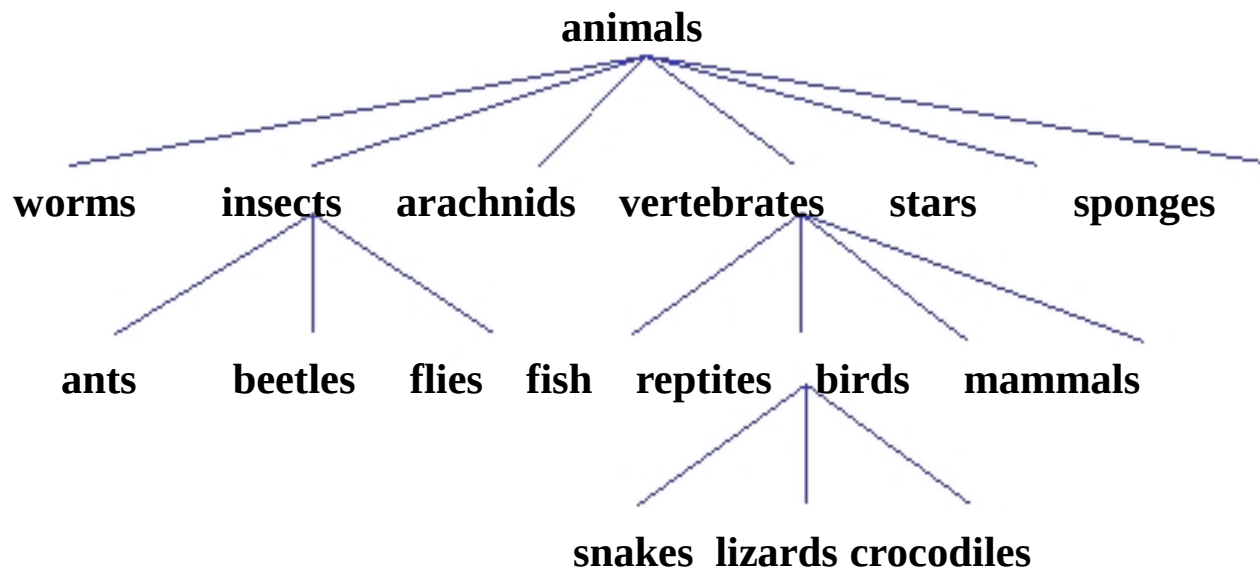
CẤU TRÚC CÂY TRONG THỰC TẾ

- Thông thường các mô hình tổ chức sẽ có cấu trúc cây:
 - Một cây tổ chức là một cây không có thứ tự có cấu trúc phân tầng, như tổ chức của phòng ban trong thương mại.



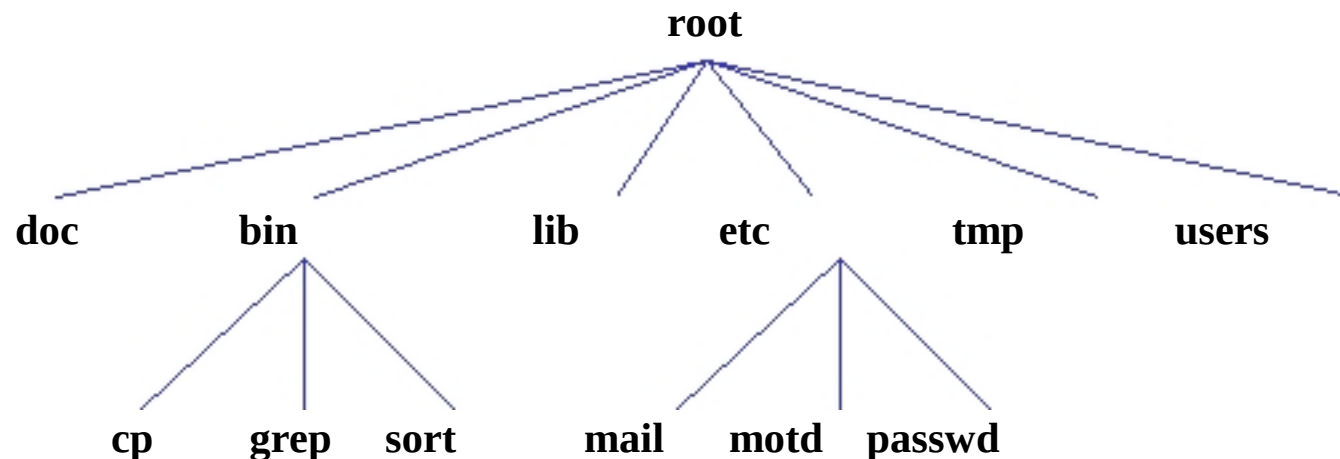
CẤU TRÚC CÂY TRONG THỰC TẾ

- Sơ đồ phân loại sinh học cũng là một loại cây phổ biến.
 - Cây này cũng không có thứ tự nhưng có tổ chức phân lớp.



CẤU TRÚC CÂY TRONG THỰC TẾ

- Tổ chức file và các thư mục
 - Chúng ta có thể mô hình hóa tổ chức các file bằng cách dùng cây không sắp xếp theo mô hình nút lá, và các thư mục như các nút cha.





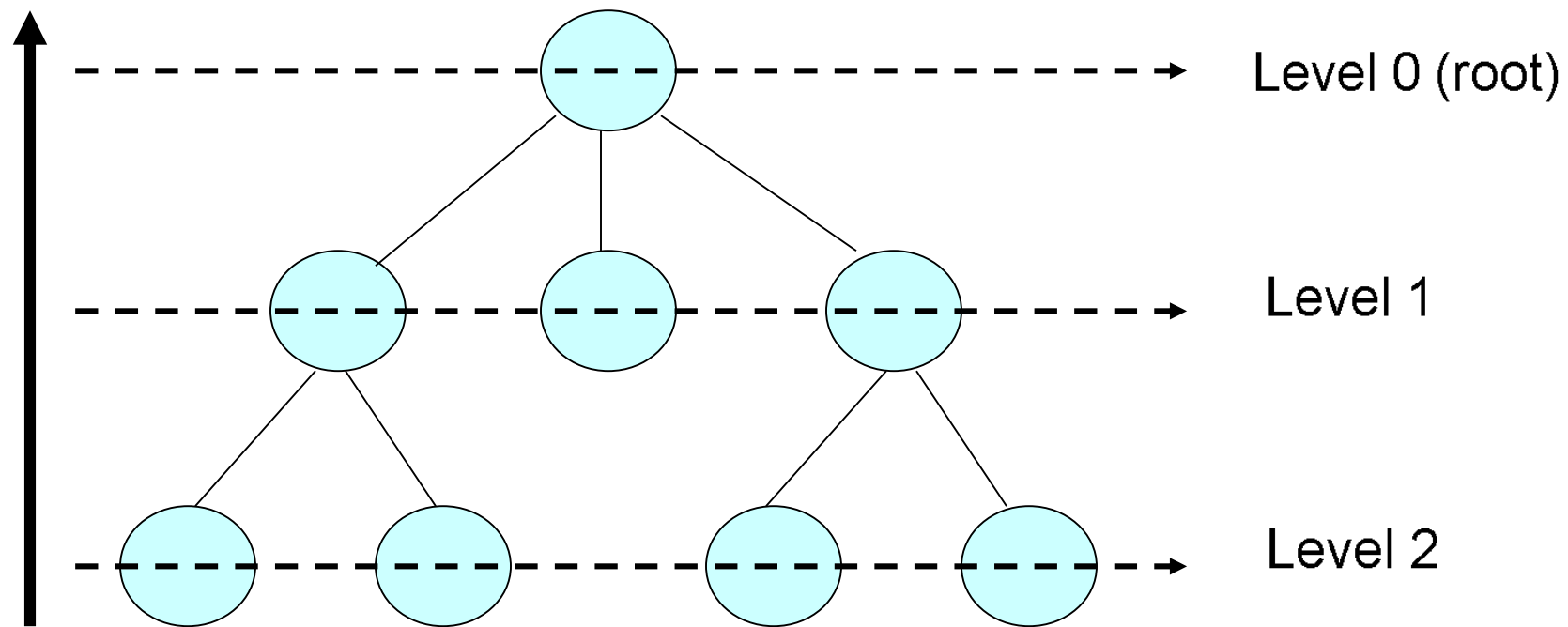
Thao tác trên cấu trúc cây

Nó phải có thể

- truy cập vào nút gốc của cây
- truy cập tất cả các tổ tiên của một nút trong cây
- truy cập tất cả con cháu của một nút trong cây
- thêm một nút mới vào cây,
- xóa một nút nhất định khỏi cây
- Duyệt cây.

Một số định nghĩa

height = 3





Một số định nghĩa

- Cây có thể được sắp xếp hoặc không được sắp xếp.
- Cây không có thứ tự, cây là cây ở hình dạng. Theo nghĩa cấu trúc, nó trông giống như một cái cây.
 - Đối với bất kỳ nút nhất định nào, không có thứ tự nào được áp đặt cho các nút con của nút đó



Một số định nghĩa

- Một cây có thứ tự áp đặt thứ tự trên các nút
- Có nhiều chiến lược sắp xếp thứ tự, nhưng theo nghĩa đơn giản nhất, thứ tự có thể được áp đặt bằng cách gán các số khác nhau cho các nút con của nút
- Lưu ý rằng thứ tự và duyệt cây không giống nhau

1

1

2

3



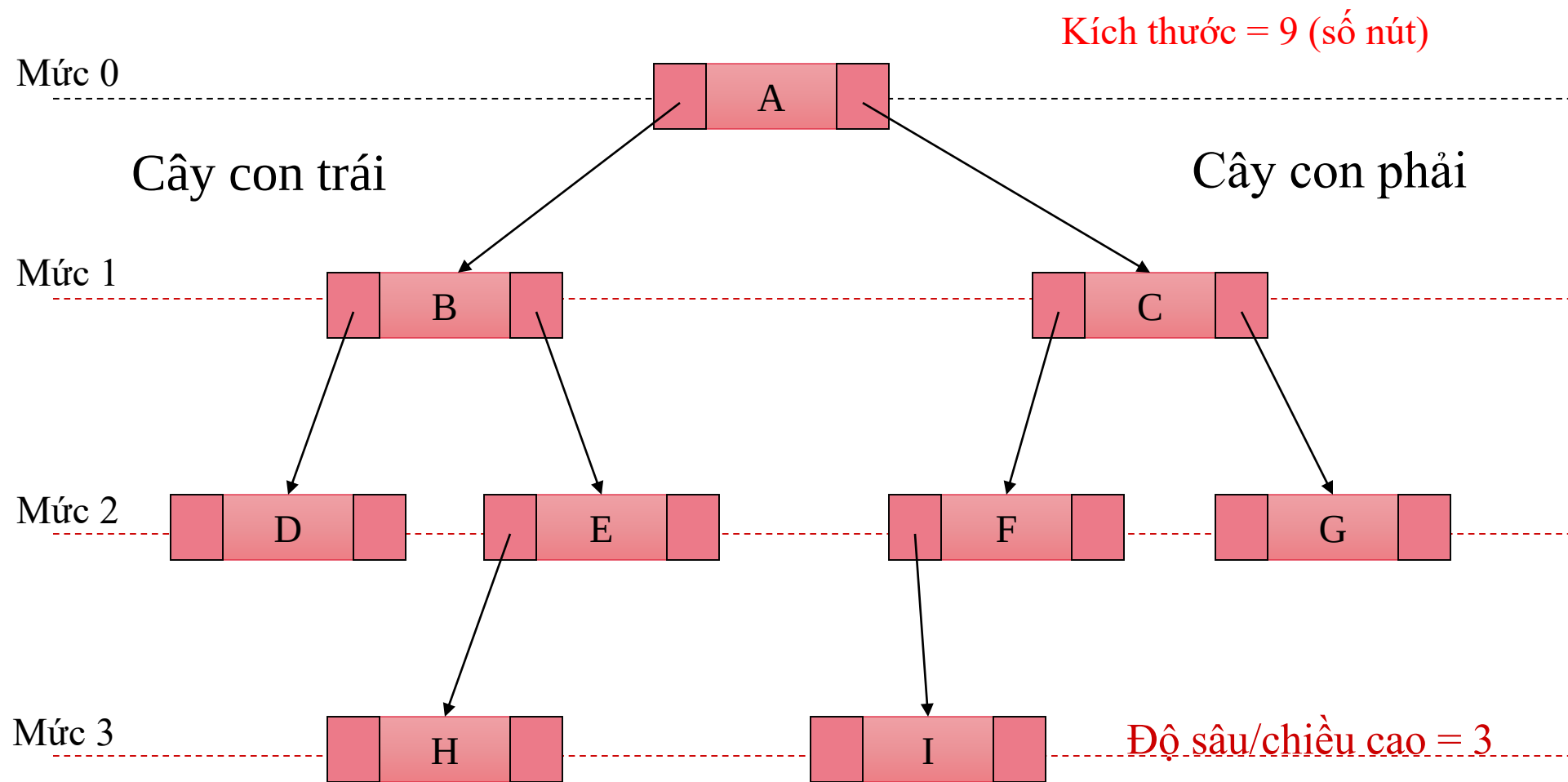
Một số định nghĩa

- Cây có thể cân bằng hoặc không cân bằng
- Cây cân bằng tốt là cây không có nút nào xa gốc hơn bất kỳ nút nào khác
- Các giải thuật cân bằng khác nhau cho phép các định nghĩa khác nhau về "xa hơn nhiều" và đi kèm với các chi phí khác nhau để giữ cho cây cân bằng

BT – Mô tả dữ liệu

- Cấu trúc cây đơn giản nhất, mỗi nút có tối đa 2 nút con
- Tại mỗi nút gồm các 3 thành phần
 - Phần data: chứa giá trị, thông tin...
 - Liên kết đến nút con trái (nếu có)
 - Liên kết đến nút con phải (nếu có)
- Cây nhị phân có thể rỗng (ko có nút nào)
- Cây NP khác rỗng có 1 nút gốc
 - Có duy nhất 1 đường đi từ gốc đến 1 nút
 - Nút không có nút con bên trái và con bên phải là nút lá

BT – Mô tả dữ liệu





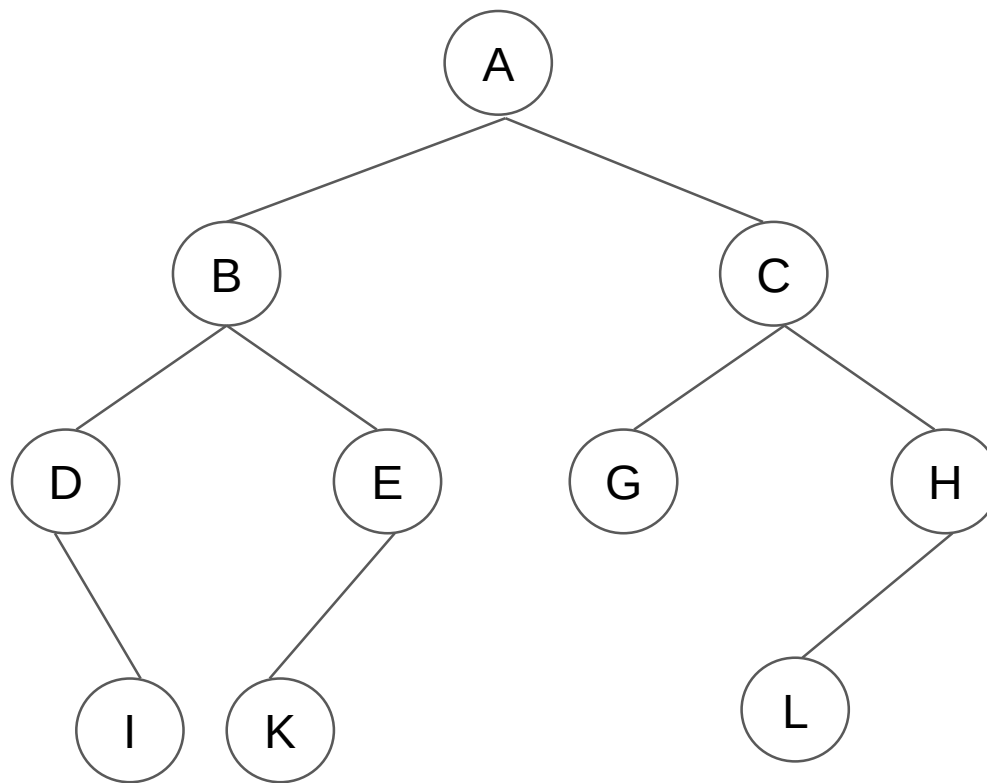
1. Khởi tạo cây
2. Kiểm tra rỗng
3. Tạo nút
4. Thêm trái
5. Thêm phải
6. Xóa trái
7. Xóa phải
8. Duyệt cây
9. Tìm kiếm
10. Xóa cây

BT- Duyệt cây

- Duyệt cây:
 - Do cây là cấu trúc không tuyến tính
 - 3 cách duyệt cây NP
 - Duyệt theo thứ tự trước PreOrder: NLR
 - Duyệt theo thứ tự giữa InOrder: LNR
 - Duyệt theo thứ tự sau PostOrder: LRN

7.3.3. BT- Duyệt cây

- Ví dụ:



PreOrder : A, B, D, I, E, K, C, G, H, L

InOrder : D, I, B, K, E, A, G, C, L, H

PostOrder : I, D, K, E, B, G, L, H, C, A

Hiện thực cây nhị phân

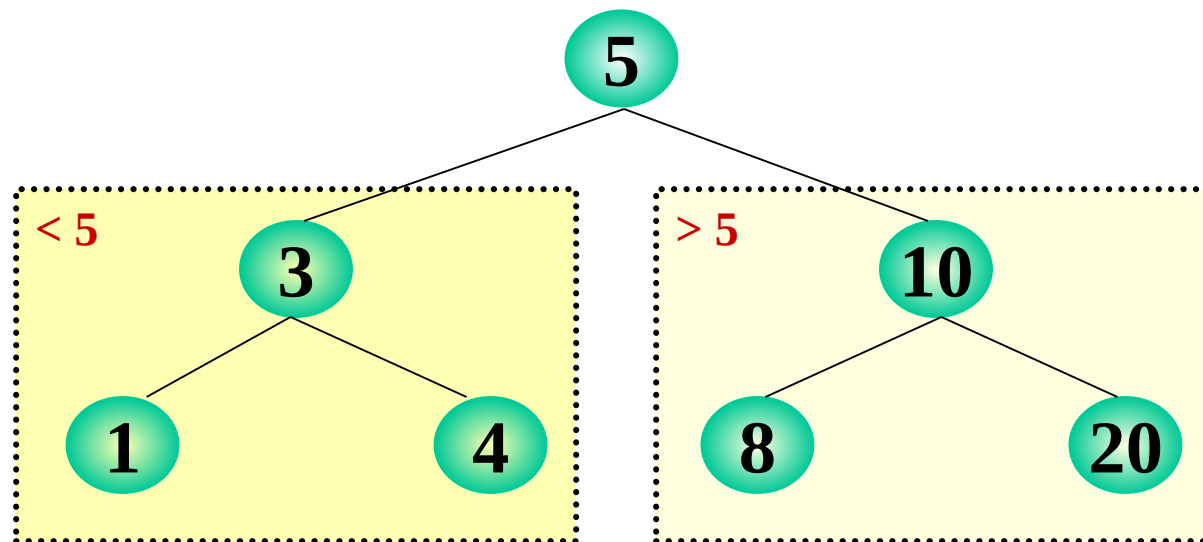
- Các thao tác mở rộng khác:
 - Đếm số nút lá: CountLeaf
 - Đếm số nút trên cây: CountNode
 - Xác định độ sâu/chiều cao của cây
 - Tìm giá trị nhỏ nhất/lớn nhất trên cây
 - Tính tổng các giá trị trên cây
 - Đếm số nút có giá trị bằng x

Cho trước 1 mảng a có n phần tử (mảng số nguyên/ hoặc mảng cấu trúc có một trường là khóa), hãy tạo một cây nhị phân có n node, mỗi nút lưu 1 phần tử của mảng.

1. Cài đặt hàm duyệt cây theo thứ tự: LNR, NLR, LRN, mức.
2. Tìm node có giá trị là X .
3. Xác định chiều cao của cây
4. Đếm số node trên cây.
5. Đếm số node lá

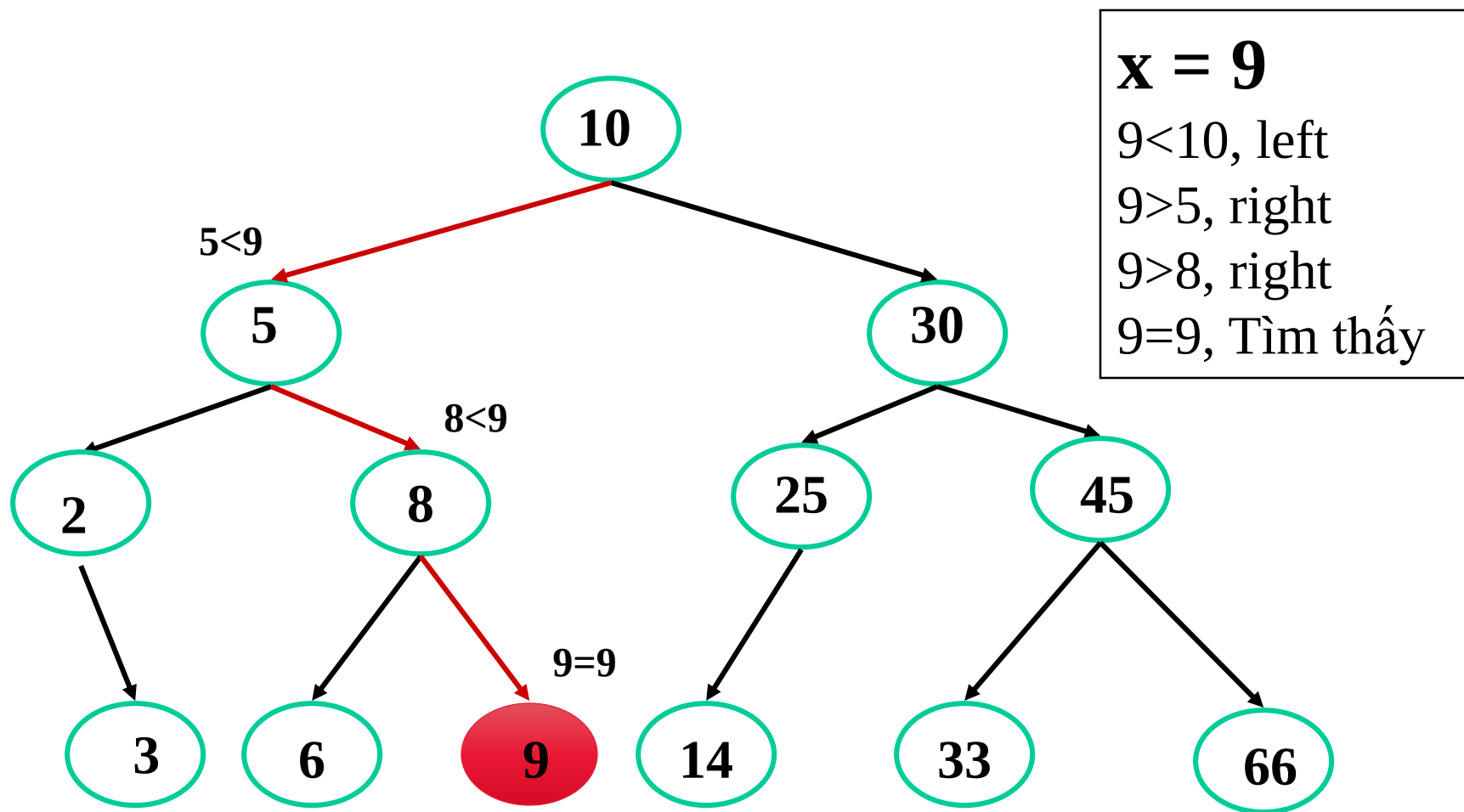
BST – Khái niệm

- BST là cây nhị phân mà **mỗi nút thoả**
 - Giá trị của tất cả nút con trái $<$ giá trị của nút đó
 - Giá trị của tất cả nút con phải $>$ giá trị của nút đó



BST – Cài đặt

- Thao tác tìm kiếm



BST – Cài đặt

■ Search

• Xuất phát từ gốc

- Nếu nút = NULL => ko tìm thấy
- Nếu khoá x = khoá nút gốc => tìm thấy
- Ngược lại nếu khoá x < khoá nút gốc => **Tìm trên cây bên trái**
- Ngược lại => **Tìm trên cây bên phải**

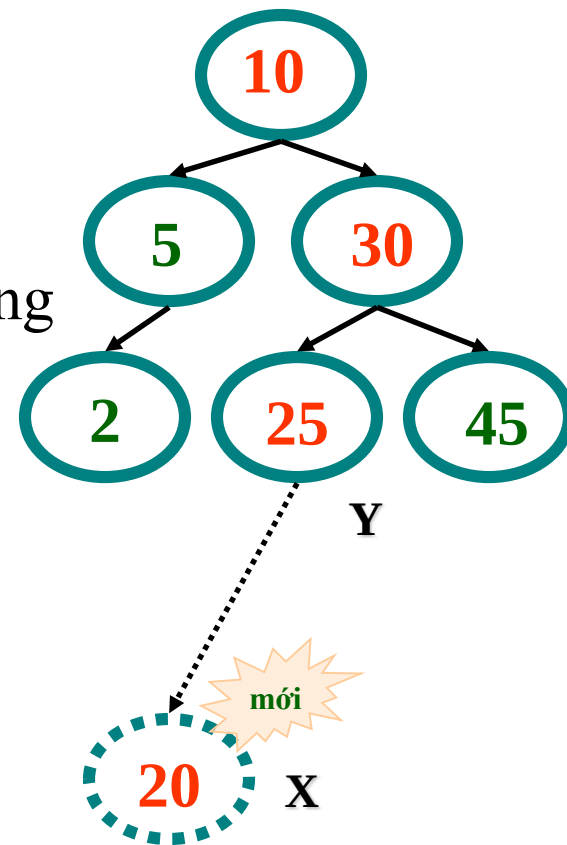
```
Node* Search(Node* pTree, int x){...}
```

BST – Cài đặt

- Xây dựng cây BST
 - Chèn
 - Xóa
- Luôn duy trì tính chất
 - Giá trị nhỏ hơn ở bên cây con phải
 - Giá trị lớn hơn ở bên cây con trái

BST – Cài đặt

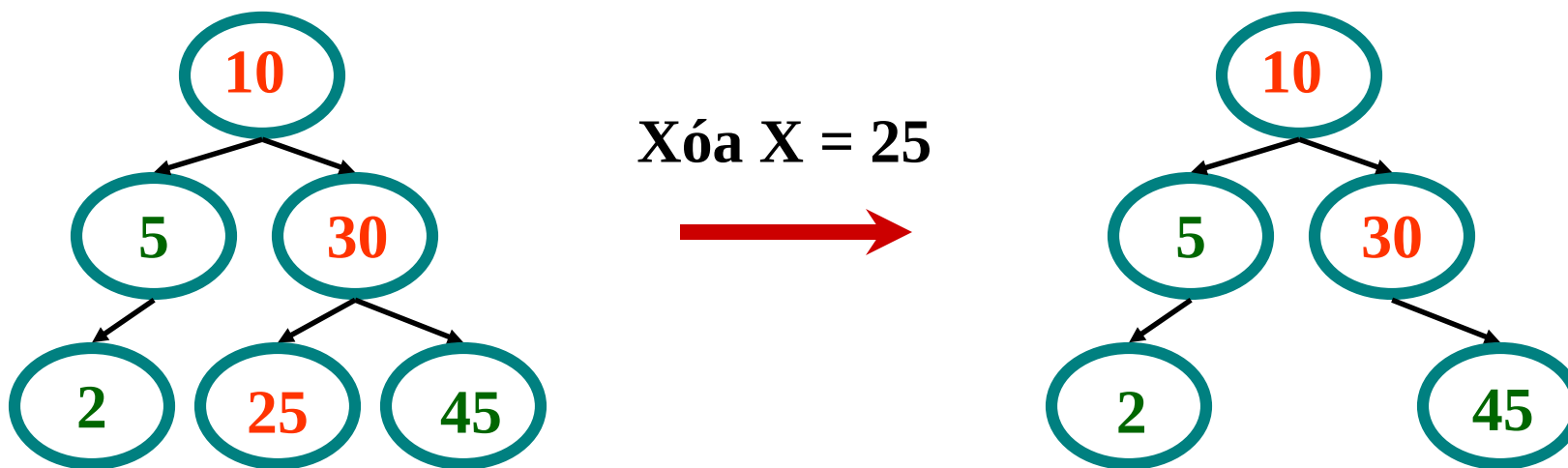
- Thêm một nút có dữ liệu x vào cây
- Nếu cây rỗng $\rightarrow$ thêm trực tiếp
- Ngoài ra:
 - Thực hiện tìm kiếm giá trị x
 - Tìm đến cuối nút Y (nếu x ko tồn tại trong cây)
 - Nếu $x < y$, thêm nút lá x bên trái của Y
 - Nếu $x > y$, thêm nút lá x bên phải của Y



- Delete: xóa nhưng phải đảm bảo vẫn là cây BST
 - Thực hiện tìm nút có giá trị x
 - Nếu nút là nút lá, delete nút
 - Ngược lại
 - Thay thế nút bằng **một trong hai** nút sau
 - Y là nút lớn nhất của cây con bên trái
 - Z là nút nhỏ nhất của cây con bên phải
 - Chọn nút Y hoặc Z để thế chỗ
 - Giải phóng nút

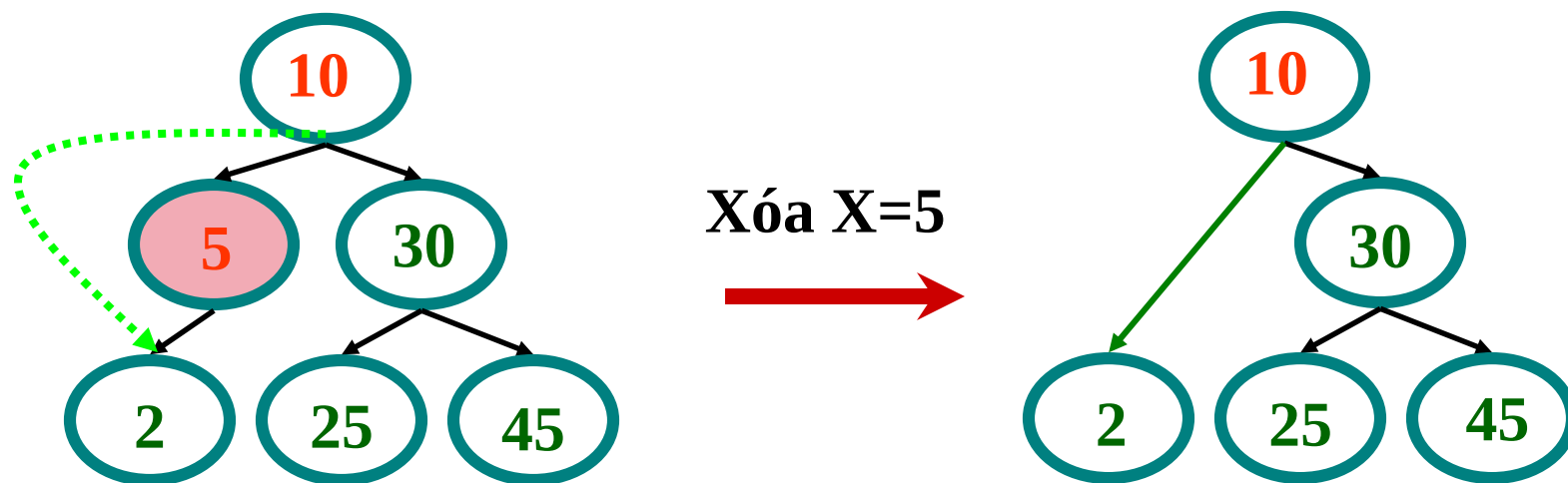
BST – Cài đặt - Delete

- TH 1: nút p là nút lá, xoá bình thường



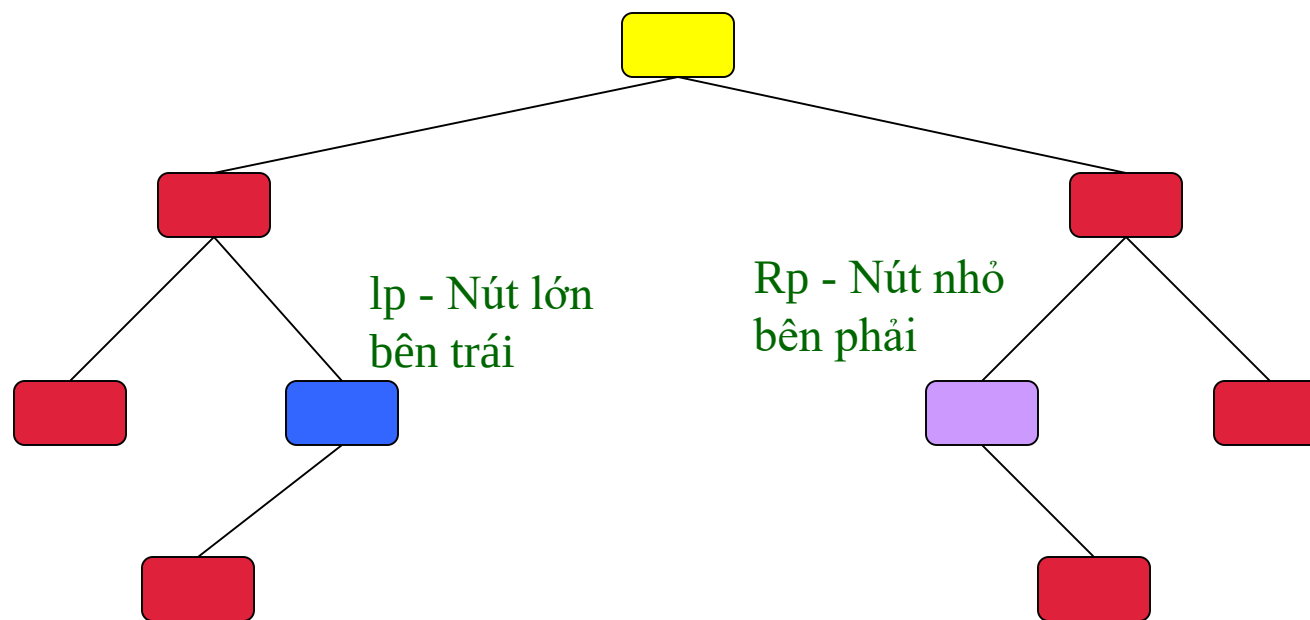
BST – Cài đặt – Delete

- **TH2:** p chỉ có 1 cây con
 - Cho **nút cha của p** trở tới **nút con duy nhất của p**
 - Hủy p



BST – Cài đặt – Delete

- **TH3:** nút p có 2 cây con, chọn nút thay thế để xóa theo 1 trong 2 cách sau
 - Nút lớn nhất trong cây con bên trái ($lp \rightarrow \text{right} == \text{NULL}$)
 - Nút nhỏ nhất trong cây con bên phải ($rp \rightarrow \text{left} == \text{NULL}$)



CÂU HỎI

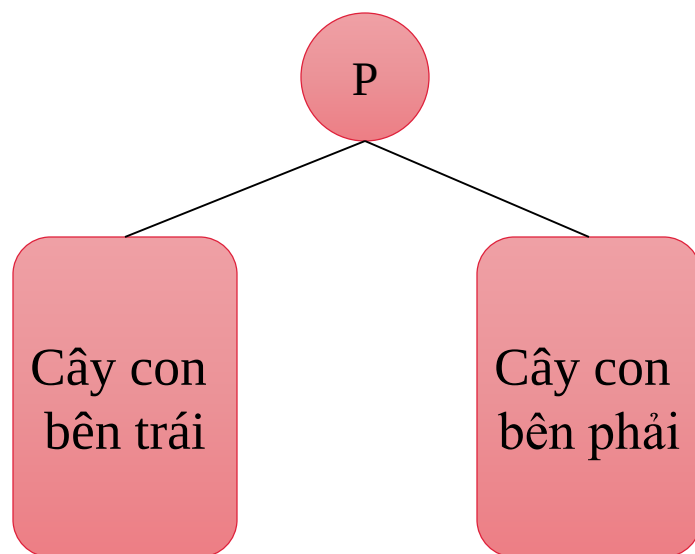
1. Cài đặt cấu trúc dữ liệu liên kết cho cây nhị phân tìm kiếm
2. Cài đặt các thao tác xây dựng cây: Init, IsEmpty, CreateNode
3. Cài đặt thao tác cập nhật: Insert, Remove, ClearTree
4. Xuất danh sách tăng dần và giảm dần

CÂY NHỊ PHÂN TÌM KIẾM CÂN BẰNG

- Do hai nhà toán học người Nga là **Adelson Velski** và **Landis** xây dựng vào năm 1962 nên còn được gọi là cây AVL.
- Nội dung
 - 9.1 Định nghĩa
 - 9.2 Các tác vụ xoay
 - 9.3 Thêm một nút vào cây AVL
 - 9.4 Cài đặt cây AVL

Định nghĩa

- Cây NPTK cân bằng
 - Là cây nhị phân tìm kiếm
 - Tại mỗi nút: số nút trên nhánh trái và nhánh phải chênh lệch ko quá 1 nút!



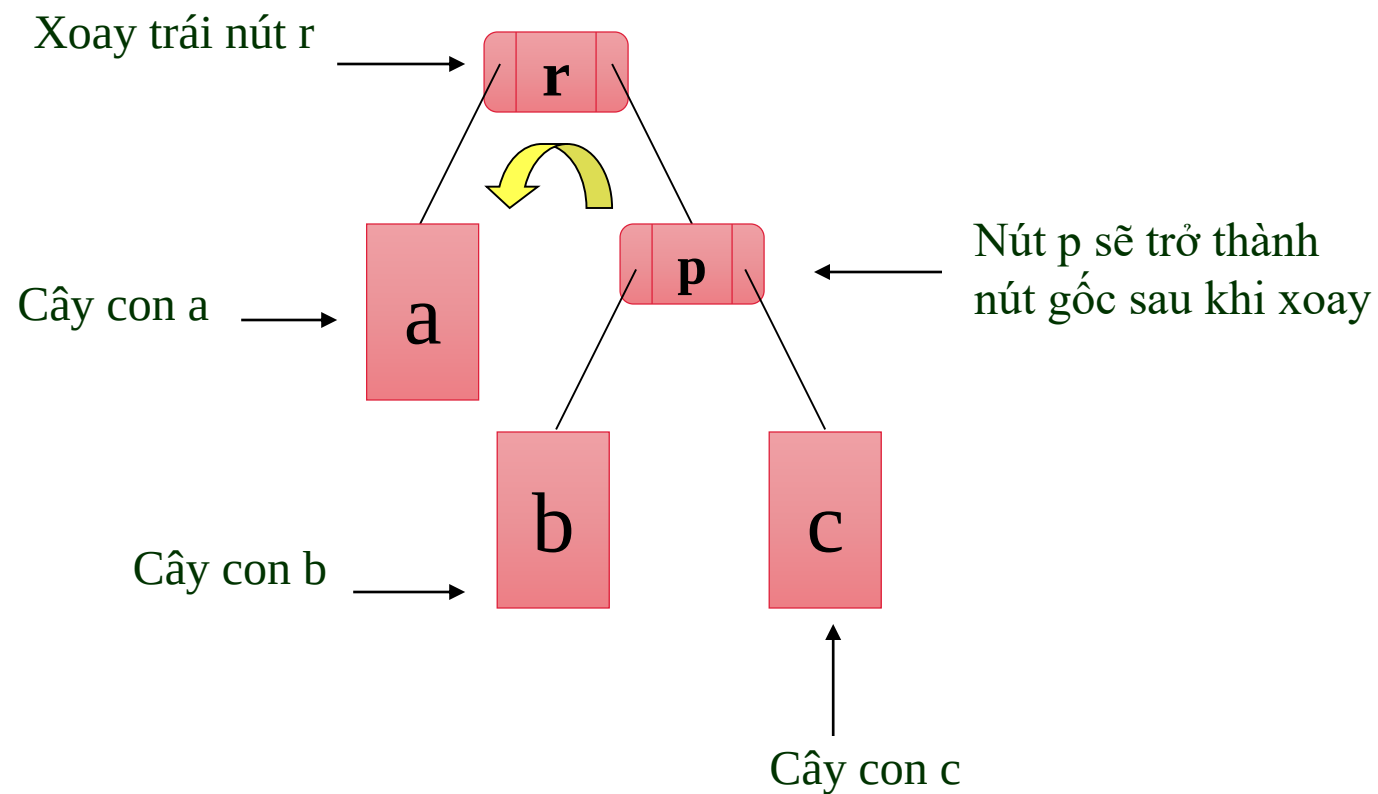
$nL(p)$: số nút cây con trái nút p
 $nR(p)$: số nút cây con phải nút p

Các tác vụ xoay

- Quá trình cập nhật cây nhị phân tìm kiếm thường làm cây mất cân bằng
- Thao tác:
 - Xoay trái RotateLeft
 - Xoay phải RotateRight

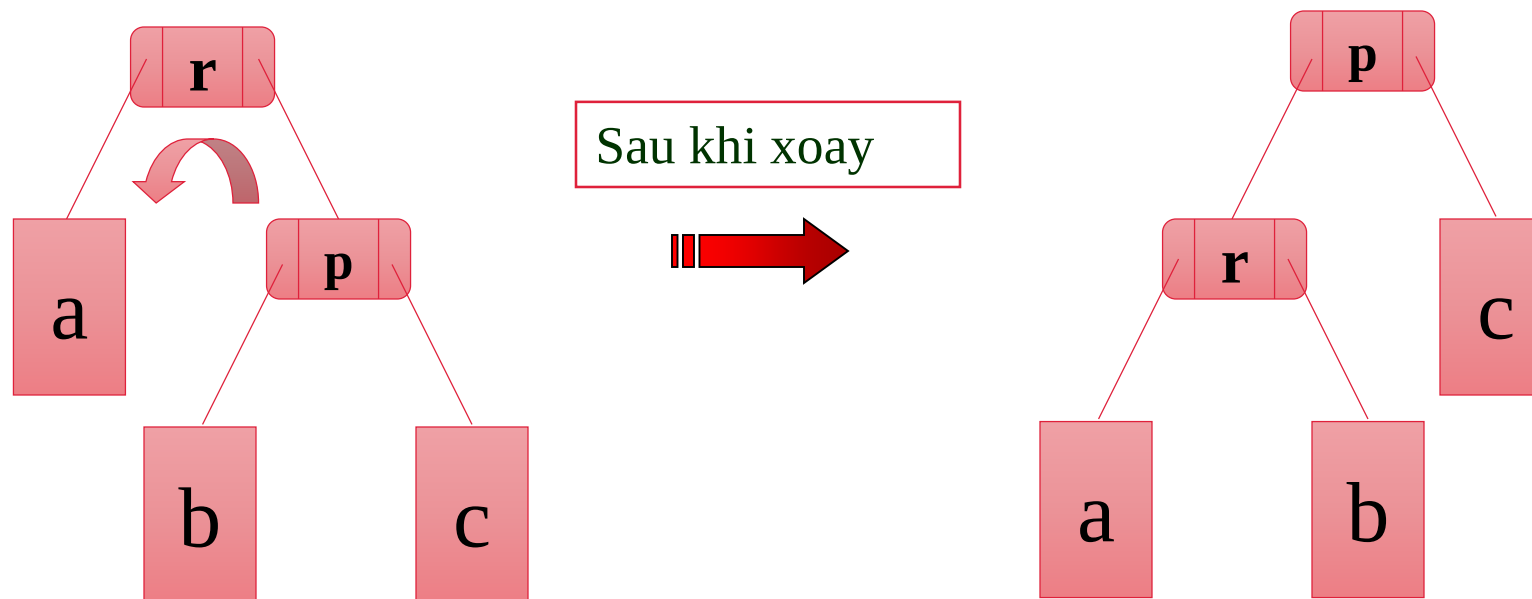
Các tác vụ xoay

- RotateLeft



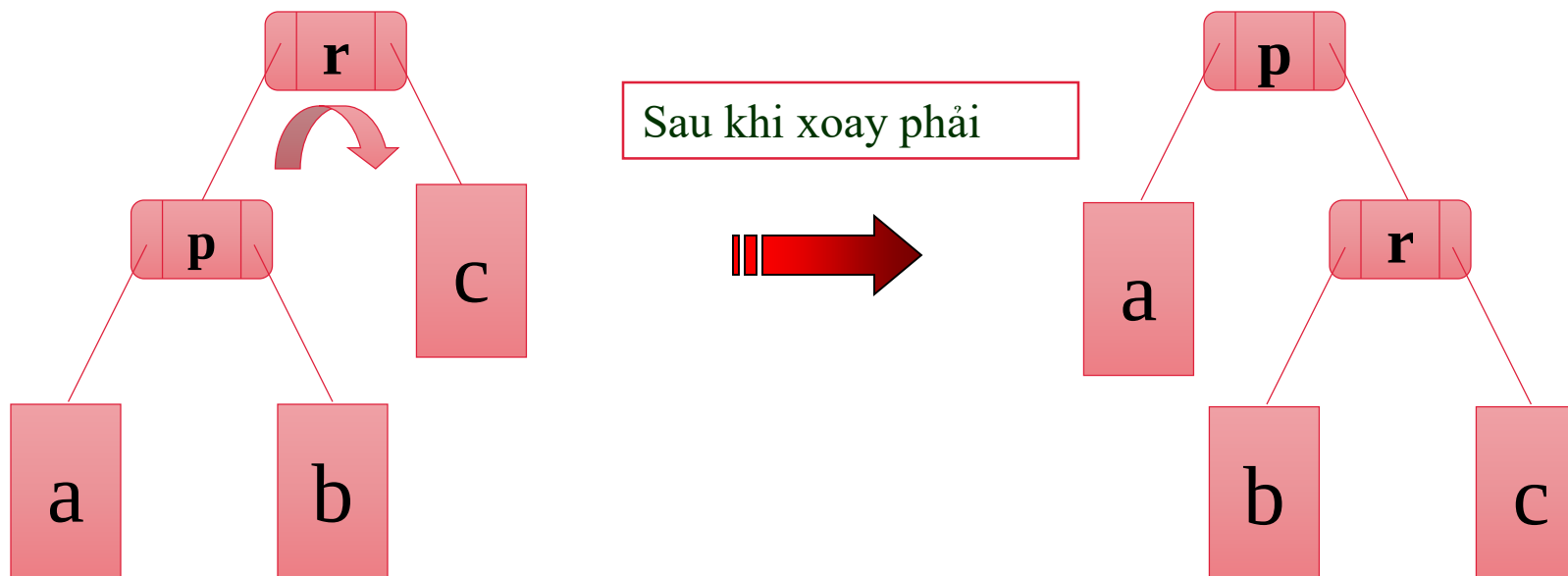
Các tác vụ xoay

- RotateLeft



Các tác vụ xoay

- RotateRight

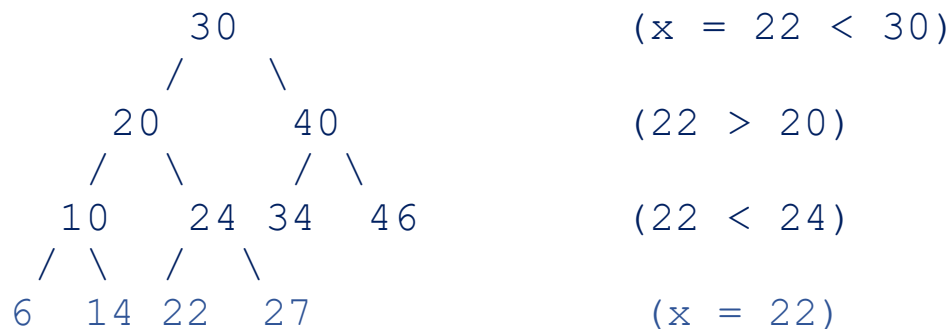


BÀI TẬP

1. Xét tác vụ insert và remove để thêm nút và xoá nút trên cây BST.
 - Vẽ lại hình ảnh của cây BST nếu thêm các nút vào cây theo thứ tự như sau: 8, 3, 5, 2, 20, 11, 30, 9, 18, 4.
 - Vẽ lại hình ảnh của cây trên nếu ta lần lượt xoá 2 nút 5 và 20.
2. Cài đặt cấu trúc dữ liệu liên kết cho cây BST, với các thao tác:
 - Cài đặt các thao tác xây dựng cây: Init, IsEmpty, CreateNode
 - Cài đặt thao tác cập nhật: Insert, Remove

BÀI TẬP

Ví dụ: Cho cây nhị phân như sau



Tìm kiếm node có giá trị $x = 22$.

Thực hiện các bước như sau:

Bước 1: Bắt đầu, từ node gốc có giá trị bằng 30. Do $22 < 30$, nên node cần tìm phía cây con bên trái;

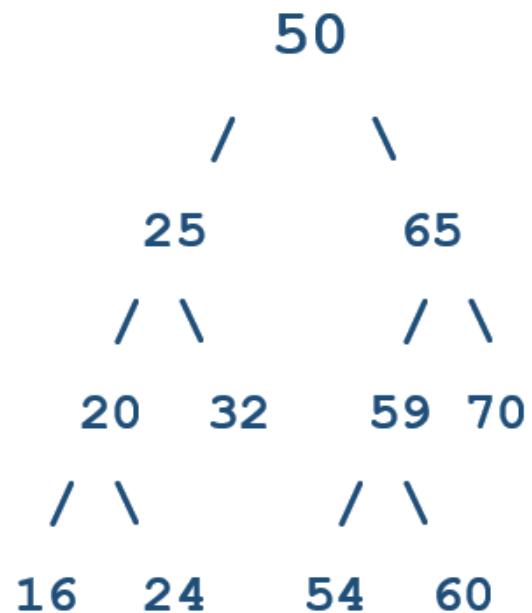
Bước 2: Node gốc của cây con bên trái bằng 20, do $22 > 20$, nên node cần tìm bên phải cây con này;

Bước 3: Node tiếp theo bằng 24, do $22 < 24$, nên node cần tìm bên trái cây con;

Bước 4: Node này có giá trị bằng $x=22$, nên ta thu được node cần tìm trong cây nhị phân.

BÀI TẬP

Bài tập: Cho cây nhị phân như sau



- a) Tìm kiếm node có giá trị $x = 24$.
- b) Tìm kiếm node có giá trị $x = 55$.
- c) Lập cây nhị phân gồm mức 0, mức 1, mức 2, mức 3.

BÀI TẬP

1. Phương pháp chia

$$h(x) = x \% M$$

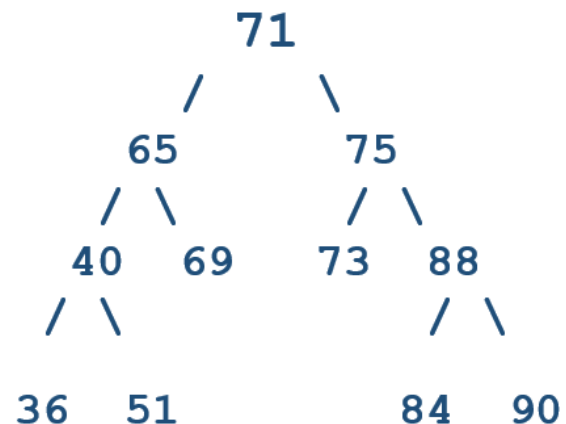
a) Tính giá trị băm cho các khóa 1533 ; 3471, 1363, 2564

$M = 5$ (Chọn số lẻ - kích thước bảng)

b) Tính giá trị băm cho các khóa 29, 18, 40, 15, 32, 59

$M = 7$ (Chọn số lẻ - kích thước bảng)

2. Cho cây nhị phân như sau



a) Tìm kiếm node có giá trị $x = 85$.

b) Lập cây nhị phân gồm mức 0, mức 1, mức 2, mức 3.

#Thực hành 01:

```
>>> class treeNode:
    def __init__(self, val = None):
        self.val = val
        self.left = None
        self.right = None

>>> root = treeNode(10)
>>> root.left = treeNode(7)
>>> root.right = treeNode(13)
>>> root.left.left = treeNode(5)
>>> root.left.right = treeNode(8)
>>> print(root.left.right.val)    # -> 8
8
>>> print(root.left.left.val)     # -> 5
5
>>> # #      10
# #    /  \
# #   7    13

>>> # #      10
# #    /  \
# #   7    13
# #  /  \
# # 5    8
```

#Thực hành 02:

```
>>> class BinaryTreeNode:
    def __init__(self, data):
        self.data = data
        self.leftChild = None
        self.rightChild=None

>>> def insert(root,newValue):

    if root is None:
        root=BinaryTreeNode(newValue)
        return root
    if newValue<root.data:
        root.leftChild=insert(root.leftChild,newValue)
    else:
        root.rightChild=insert(root.rightChild,newValue)
    return root

>>> root= insert(None,15)
>>> insert(root,10)
>>> insert(root,25)
>>> insert(root,6)
>>> insert(root,14)
>>> insert(root,20)
>>> insert(root,60)
>>> a1=root
>>> a2=a1.leftChild
>>> a3=a1.rightChild
>>> a4=a2.leftChild
>>> a5=a2.rightChild
```


#Thực hành 02:

```
>>> a6=a3.leftChild
>>> a7=a3.rightChild
>>> print("Root Node is:")
Root Node is:
>>> print(a1.data)
15
>>> print("left child of node is:")
left child of node is:
>>> print(a1.leftChild.data)
10
>>> print("right child of node is:")
right child of node is:
>>> print(a1.rightChild.data)
25
>>> print("Node is:")
Node is:
>>> print(a2.data)
10
>>> print("left child of node is:")
left child of node is:
>>> print(a2.leftChild.data)
6
>>> print("right child of node is:")
right child of node is:
>>> print(a2.rightChild.data)
14
>>> print("Node is:")
Node is:
>>> print(a3.data)
25
```

#Thực hành 02:

```
>>> print("left child of node is:")
left child of node is:
>>> print(a3.leftChild.data)
20
>>> print("right child of node is:")
right child of node is:
>>> print(a3.rightChild.data)
60

>>> print("Node is:")
Node is:
>>> print(a4.data)
6
>>> print("left child of node is:")
left child of node is:
>>> print(a4.leftChild)
None
>>> print("right child of node is:")
right child of node is:
>>> print(a4.rightChild)
None

>>> print("Node is:")
Node is:
>>> print(a5.data)
14
```

```
>>> print("left child of node is:")
left child of node is:
>>> print(a5.leftChild)
None
>>> print("right child of node is:")
right child of node is:
>>> print(a5.rightChild)
None

>>> print("Node is:")
Node is:
>>> print(a6.data)
20
>>> print("left child of node is:")
left child of node is:
>>> print(a6.leftChild)
None
>>> print("right child of node is:")
right child of node is:
>>> print(a6.rightChild)
None

>>> print("Node is:")
Node is:
>>> print(a7.data)
60
>>> print("left child of node is:")
left child of node is:
>>> print(a7.leftChild)
None
>>> print("right child of node is:")
right child of node is:
>>> print(a7.rightChild)
None
```

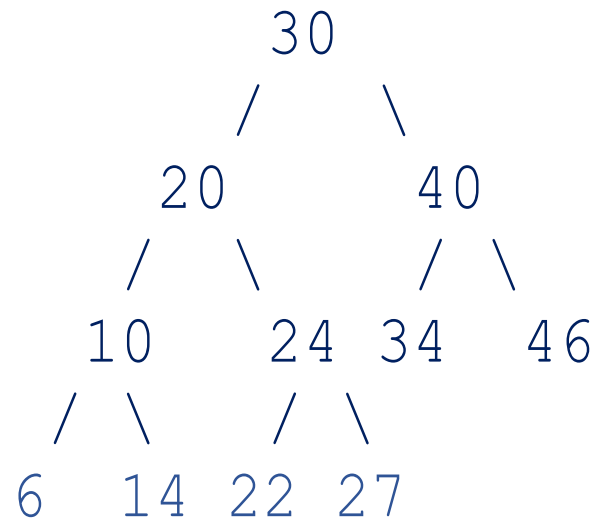
#Thực hành 02:

```
>>> # #  
      # #  
      /  \  
     10   25  
    /  \  /  \  
   6  14 20 60
```



#Thực hành 03:

#Cho cây nhị phân như sau



#Note: tương tự thực hành 02

#Thực hành 04:

#cây nhị phân tìm kiếm

```
class Node:
```

```
    def __init__(self, data):
```

```
        self.left = None
```

```
        self.right = None
```

```
        self.data = data
```

```
    #Insert to create nodes
```

```
    def insert(self, data):
```

```
        if self.data:
```

```
            if data < self.data:
```

```
                if self.left is None:
```

```
                    self.left = Node(data)
```

```
                else:
```

```
                    self.left.insert(data)
```

```
            if data > self.data:
```

```
                if self.right is None:
```

```
                    self.right = Node(data)
```

```
                else:
```

```
                    self.right.insert(data)
```

```
        else:
```

```
            self.data = data
```

```
    #find value to compare the value
```

```
    def findval(self, inval):
```

```
        if (inval < self.data):
```

```
            if self.left is None:
```

```
                return (str(inval)+" Not Found")
```

```
            return (self.left.findval(inval))
```

```
        if (inval > self.data):
```

```
            if self.right is None:
```

```
                return (str(inval)+" Not Found")
```

```
            return self.right.findval(inval)
```

```
        else:
```

```
            print(str(self.data) + ' is found')
```

```
    def PrintTree(self):
```

```
        if self.left:
```

```
            self.left.PrintTree()
```

```
            print( self.data)
```

```
        if self.right:
```

```
            self.right.PrintTree()
```

```
            print( self.data)
```

```
root = Node(15)
```

```
root.insert(10)
```

```
root.insert(19)
```

```
root.insert(5)
```

```
root.insert(13)
```

```
root.insert(14)
```

```
root.insert(29)
```

```
print(root.findval(29))
```

```
print(root.findval(10))
```

```
print(root.findval(40))
```

